

UAE E-Commerce Fraud Detection: A Supervised Machine Learning Approach

Abel Chuchu

M80009115

ICS351 Machine Learning

Zayed University

Muhammad Al-Zafar Khan (PHD)

February 22, 2026

Executive summary

This assignment focuses on supervised machine learning to predict fraudulent e-commerce transactions in the UAE. I used a dataset with 40+ features and 100,000 synthetic UAE-based transaction records, with a fraud rate of 8.2%. I engineered a focused set of six features and trained five classification models, which include Logistic Regression, Decision Tree, Random Forest, Stochastic Gradient Descent Classifier, and a Voting Ensemble. I performed hyperparameter tuning using GridSearchCV with 5-fold stratified cross-validation. For the primary evaluation metric, I used binary F1-score for the fraud class to ensure the performance reflects the model's ability to detect fraud rather than simply classify the majority class correctly. Decision Tree achieved the highest F1-score, although the other models resulted in very close results.

Section 1: Introduction and Data Exploration

1.1 Project Objective

The primary goal of my assignment is to develop a strong supervised machine learning model that can accurately predict whether an online transaction is fraudulent ($\text{is_fraud} = 1$) or legitimate ($\text{is_fraud} = 0$) based on a variety of transactional and behavioural features. I chose fraud detection because I am taking digital security and machine learning courses, and I wanted to connect these two subjects. Additionally, I wanted to train a model that has real-life use. The dataset I used is realistic, related to my field, and has a significant real-world impact, such as preventing financial losses, protecting customers, and improving trust in online platforms. The target variable, is_fraud , is a binary indicator that represents a rare but critical event. In fraud

detection, correctly identifying the minority class is far more valuable than achieving high overall accuracy.

1.2 Univariate Analysis

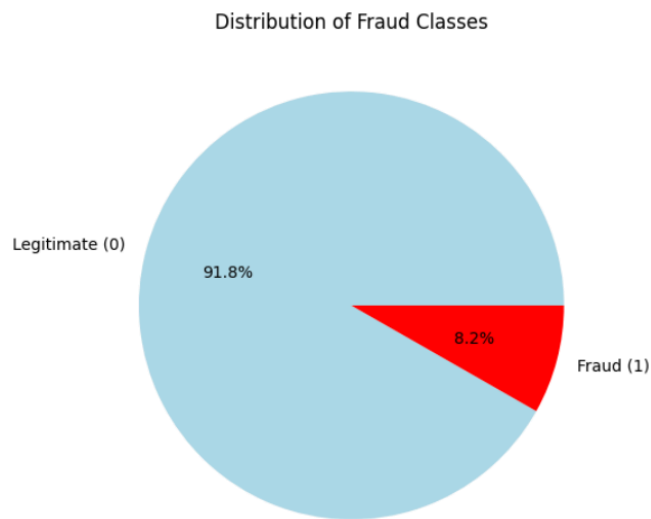


Figure 1.1: Distribution of Target Variable (is_fraud)

This pie chart shows that the dataset is highly imbalanced, as 91.8% of the transactions are legitimate, and only 8.2% are fraudulent. This imbalance is the main challenge of the project because the accuracy performance metrics will be misleading. A dummy model that predicts “Not Fraud” for every transaction would get 91.8% accuracy without catching a single fraud. This guides my use of `class_weight = ‘balanced’` in my models and the selection of binary F1-Score as my primary evaluation metric.

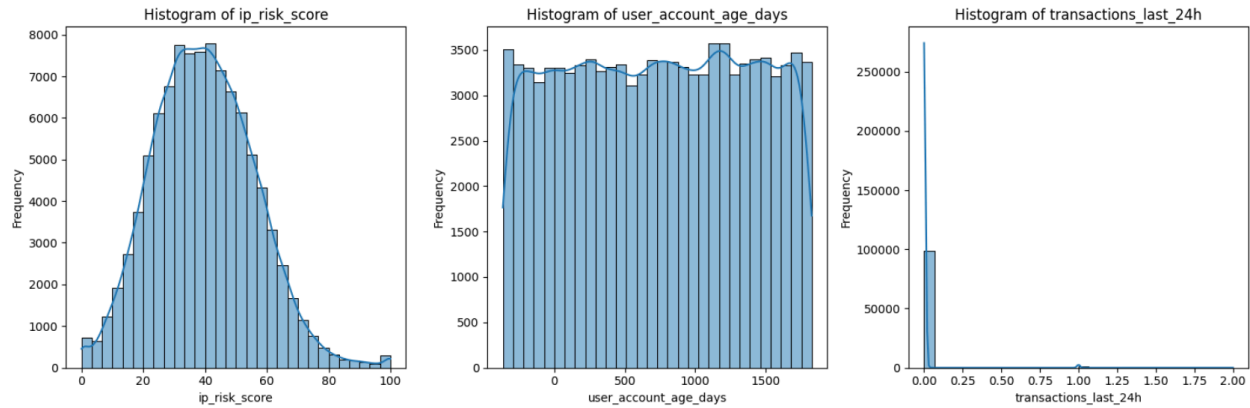
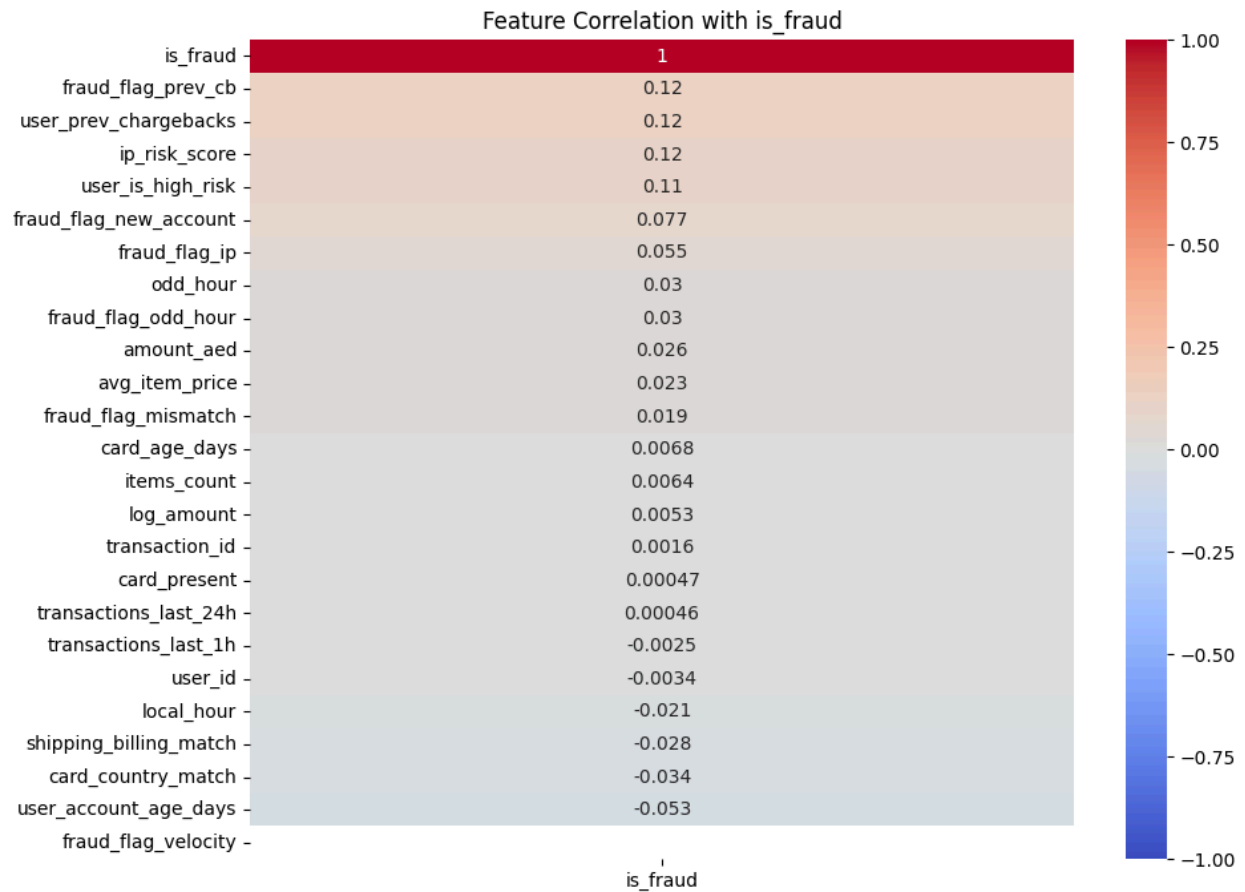


Figure 1.2: Histograms of Key Numeric Features

I plotted histograms for `ip_risk_score`, `user_account_age_days`, and `transactions_last_24h`. In `ip_risk_score`, the scores are spread widely, with some transactions having very high risk scores. This may indicate that higher scores correlate with fraud. For the second graph, the column contains negative values. This might be a placeholder for a new account. Hence, I decided to keep them as they may still carry a signal. The transactions during the last 24 hr table shows the feature is heavily zero-inflated. This means most users have a few transactions in the last day. The long tail of high values may indicate velocity-based attacks.

1.3 Bivariate Analysis



By looking at this table, I deliberately avoided features such as `fraud_flag_new_account`, `fraud_flag_ip`, and similar `fraud_flag_ip` even though they have high correlation, because those features are likely derived from predefined fraud rules. Using them could introduce data leakage, where the model indirectly learns the answer from features that already encode fraud logic. Instead, I selected original predictors such as `ip_risk_score`, `user_account_age_days`, `transactions_last_1h`, `user_prev_chargedbacks`, and `card_country_match`, which allow the model to discover patterns on its own. I also performed feature engineering by transforming `amount_aed` into `log_amount` to reduce skewness and limit the influence of extreme outliers.

```
#by Abel Chuchu
df['risk_velocity_interaction'] = (
    df['ip_risk_score'] * df['transactions_last_1h']
)
```

I tried using this and similar feature engineering, but stuck to log_amount as the new feature engineering lowered my F1-score.

By Abel Chuchu

Concept	LP Reference	Angle Covered in LP / Course Discussion	How I Applied It in This Project
Dimensionality Reduction	Session 15 - Other Dimensionality Reduction techniques	Covered from the perspective of reducing feature space to simplify visualization and potentially reduce noise using techniques such as PCA. Emphasis was placed on understanding variance explained and interpretability trade-offs.	I applied PCA only for visualization , reducing the numeric feature space to two principal components to visually inspect class separability. I did not use PCA for modelling because interpretability is critical in fraud detection, and PCA transforms features into abstract components.
Feature Engineering	Session 13 - Dimensionality Reduction techniques: Principal Component Analysis	Covered from the angle of transforming raw variables to improve model performance, handle skewness, and create meaningful predictors from domain knowledge.	I engineered log_amount using a log transformation to correct right-skewness in transaction amounts and capped it at the 99th percentile to reduce the effect of extreme outliers. I also selected behavioural features (e.g., transaction velocity, chargebacks) based on domain knowledge.
Pipelines & Scaling	Session 3 - End-to-End Machine Learning	Covered from the angle of preventing data leakage, ensuring reproducibility, and applying preprocessing consistently using Pipeline and ColumnTransformer. Scaling was discussed as necessary for linear models.	I implemented a full preprocessing pipeline combining StandardScaler for numeric features and OneHotEncoder for categorical features. This ensured that scaling was applied only to training data within cross-validation folds and prevented data leakage.
Cross-Validation	Session 4 - The Automatic Classification Task	Covered from the angle of reducing overfitting risk and obtaining a more reliable estimate of model performance using k-fold (especially stratified k-fold for imbalanced datasets).	I used Stratified 5-Fold Cross-Validation within GridSearchCV to preserve class proportions in each fold due to the 8.2% fraud rate. This provided a robust estimate of F1-score performance and reduced variance from a single train/validation split.

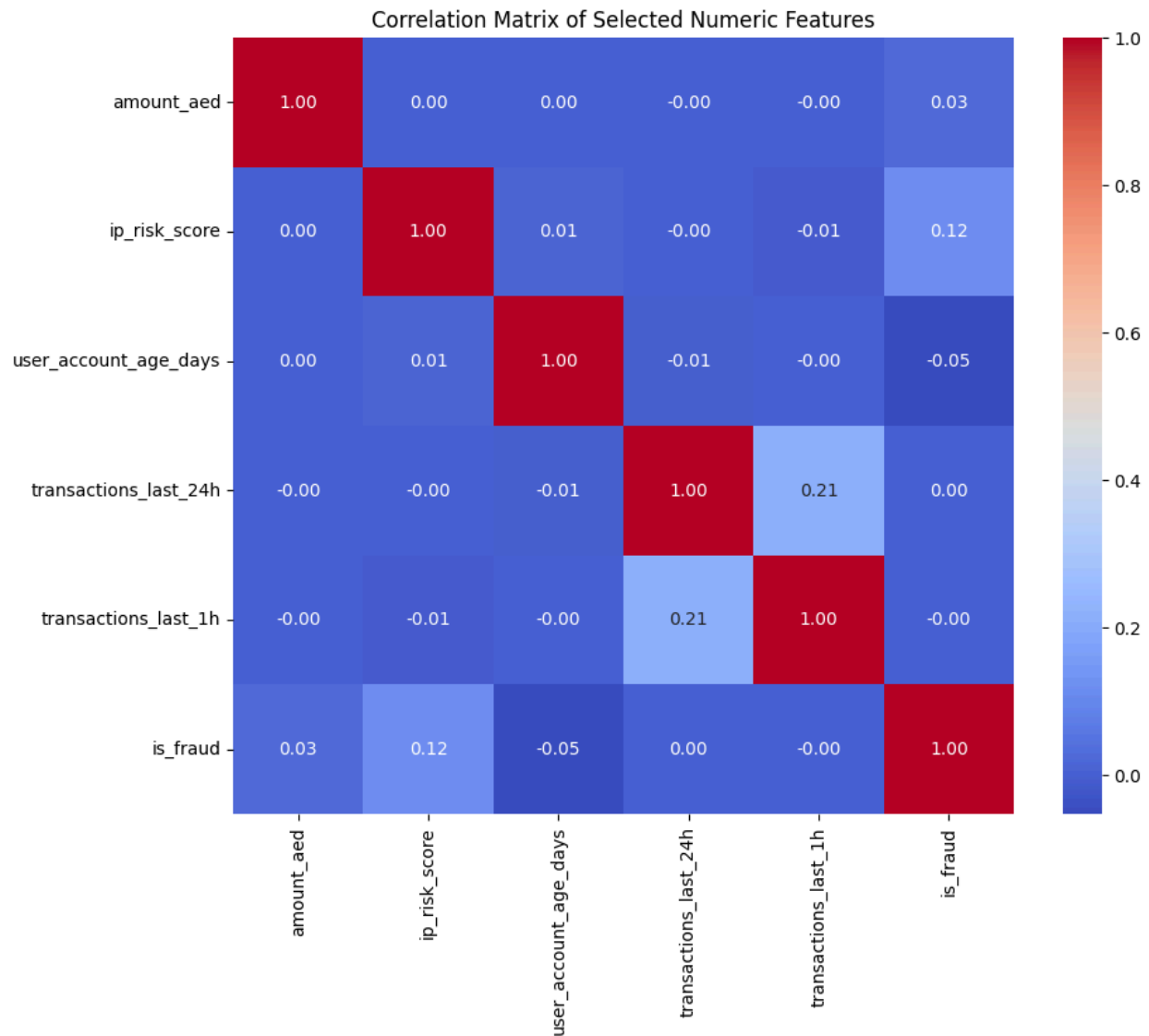


Figure 1.3: Correlation Matrix of Selected Numeric Features

From 40+ features, I kept about 7 features that are the most relevant to my context. In this correlation matrix, I learned that the correlations with `is_fraud` are weak, and the highest is `ip_risk_score` at only 0.12. I expected this result because fraud patterns are often non-linear, so a linear correlation understates their predictive power.

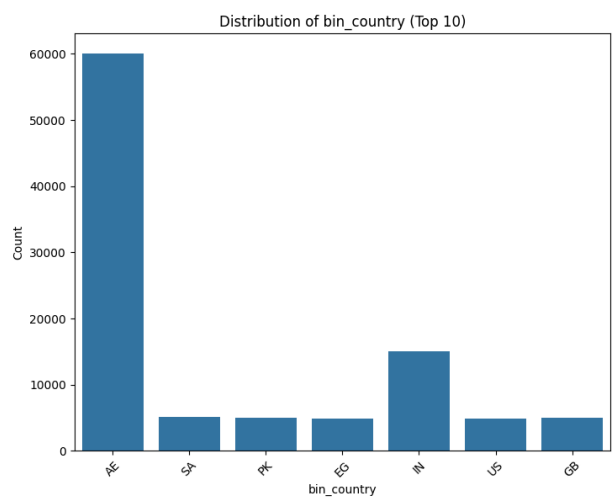
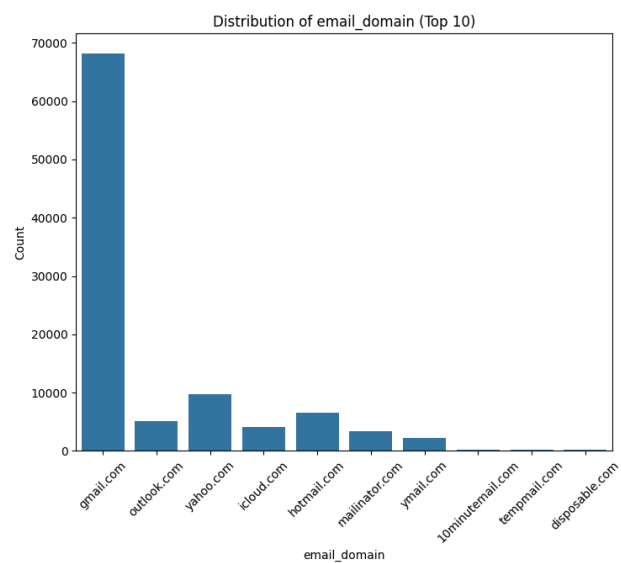
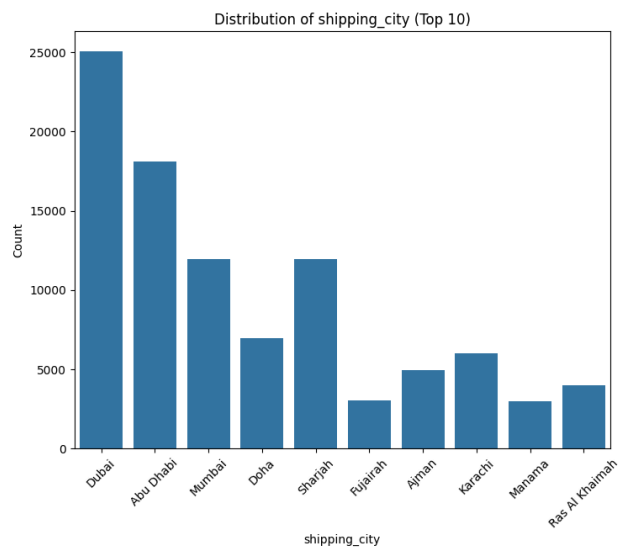
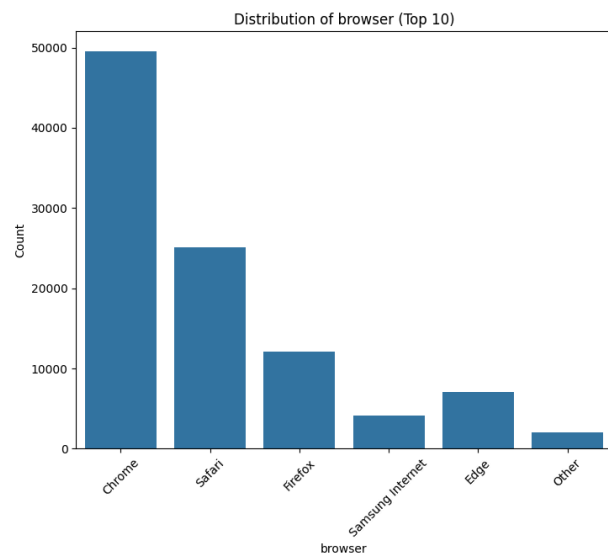
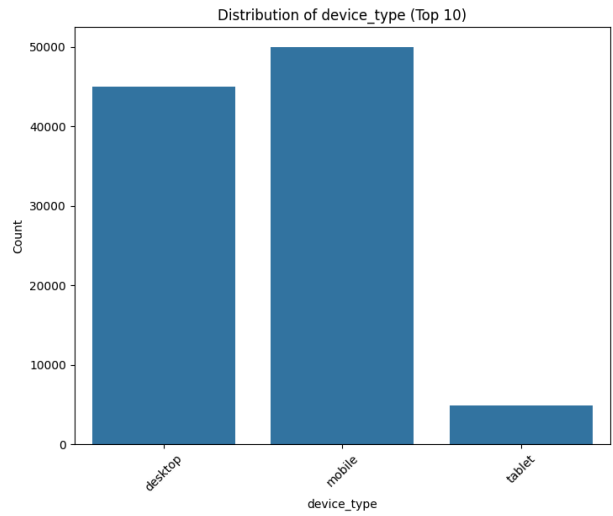
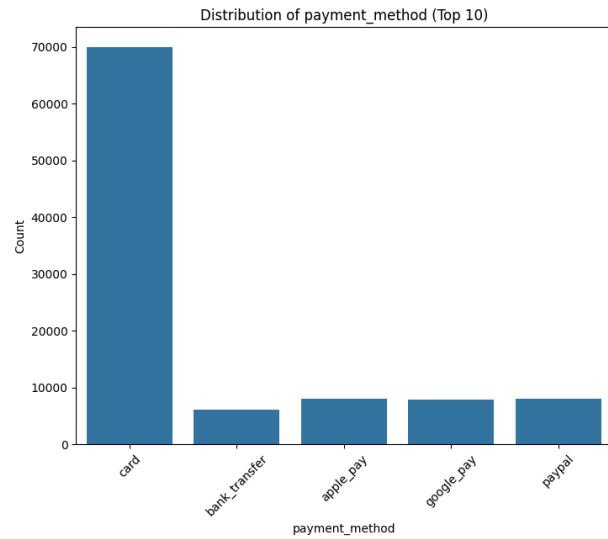


Figure 1.4: Count Plots of Categorical Features

From this count plot, I learned that card and bank transfer dominate payment methods, and the other graphs similarly show their distribution. But the main insight I got is that the categorical features contain many different categories. If I used Label Encoding, each category would be turned into a number (for example, card = 1, apple_pay = 2, paypal = 3). But this creates a fake ranking because it makes the model think PayPal > Apple Pay > card. To avoid this problem, I used One-Hot Encoding, which creates a separate binary column for each category. This treats all categories equally and prevents the model from assuming any artificial hierarchy. It is the correct method for unordered categorical variables.

Section 2: Data Cleaning, Pre-processing, and Feature Engineering

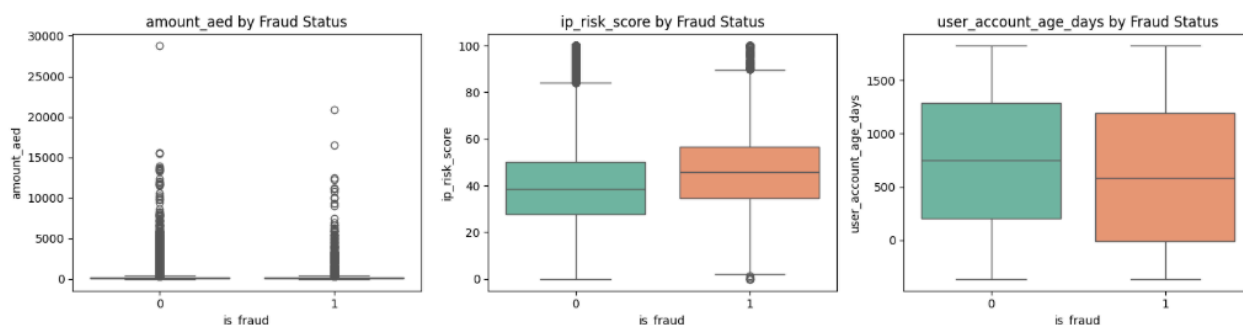
In this step, I cleaned the dataset and selected the most important features for modeling.

```
# --- 4.0: Checking for Missing Values ---
missing_values = df.isnull().sum()
missing_values = missing_values[missing_values > 0].sort_values(ascending=False)

print("--- Missing Values Check ---")
if missing_values.empty:
    print("Result: There are NO missing values in the dataset!\n")
else:
    print(missing_values)
    plt.figure(figsize=(8, 4))
    sns.barplot(x=missing_values.values, y=missing_values.index, palette='Reds_r')
    plt.title('Count of Missing Values per Column')
    plt.xlabel('Number of Missing Rows')
    plt.ylabel('Features')
    plt.show()
```

I checked for missing values again, just to show the code I learned in class. It is not needed to do it again because the dataset is synthetic and has no missing values. Cleaning

missing values is very important because even one missing value in a critical feature like `ip_risk_score` can lead to a missed fraud signal and also cause errors in training models like linear regression.



The box plots show how transaction amounts, IP risk scores, and account ages differ between legitimate and fraudulent activities. There is an outlier amount so I created a new feature called **log_amount** using log transformation(feature engineering) and capped it at the 99th percentile. I did this because there are extreme values even after using a log transformation. Having extreme outliers increases variance and affects my model, so I capped it at the 99th percentile to avoid removing the data completely.

```
# 2. Filter the dataframe to only these columns and drop any missing values just in case
df_cleaned = df[features_to_keep].dropna()

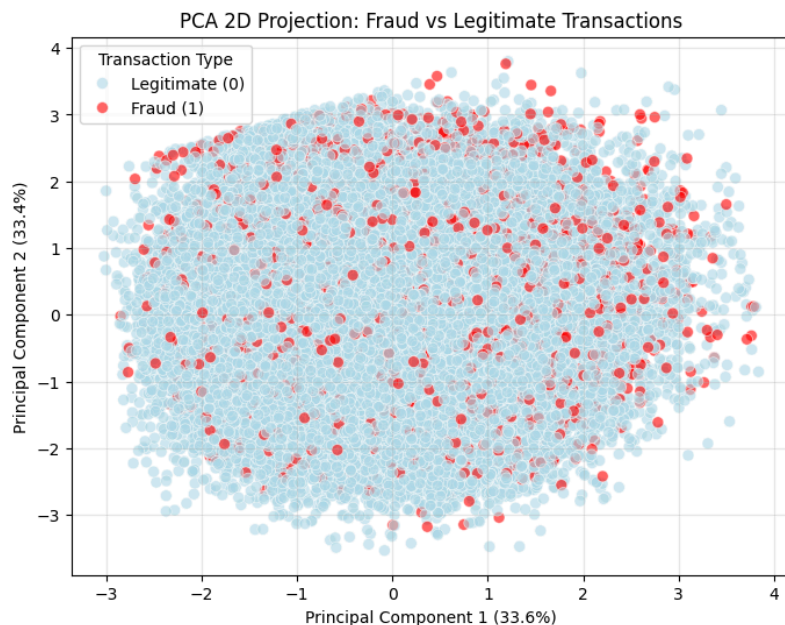
# Outlier Handling: Cap the 'log_amount' at the 99th percentile
df_cleaned['log_amount'] = np.clip(df_cleaned['log_amount'], None, df_cleaned['log_amount'].quantile(0.99))

print(f"Dataset shape after cleaning and strictly selecting 6 features: {df_cleaned.shape}")
display(df_cleaned.head())

# 4. Define X and y for modeling
X = df_cleaned.drop(columns=['is_fraud'])
y = df_cleaned['is_fraud']
```

My feature selection can be considered as dimensionality reduction. I had about 40 columns, but I reduced them to only 8. I did this to reduce noise, reduce overfitting, and make

the model faster. I didn't use Principal Component Analysis (PCA) for modelling because I wanted to show which feature caused the transaction to be flagged. If I used PCA, the features would be abstract and have no explainability. Also, from the correlation matrix, there is no strong multicollinearity, so PCA is unnecessary for redundancy removal. However, I used PCA only for visualization to show my understanding where I compressed my 4 numeric features down into 2 principal components to plot them on a 2D scatter plot and visually inspect if the fraud classes are separable.



Section 3: Data Modelling and Data Splitting

This is a supervised binary classification problem as I am using labelled data (x,y) to learn and predict fraud. I split the data into 80% training and 20% testing sets using stratified splitting to keep the fraud ratio (~8%) the same in both sets. I used stratified splitting because the dataset is highly imbalanced. If I randomly split the data without stratification, one split might

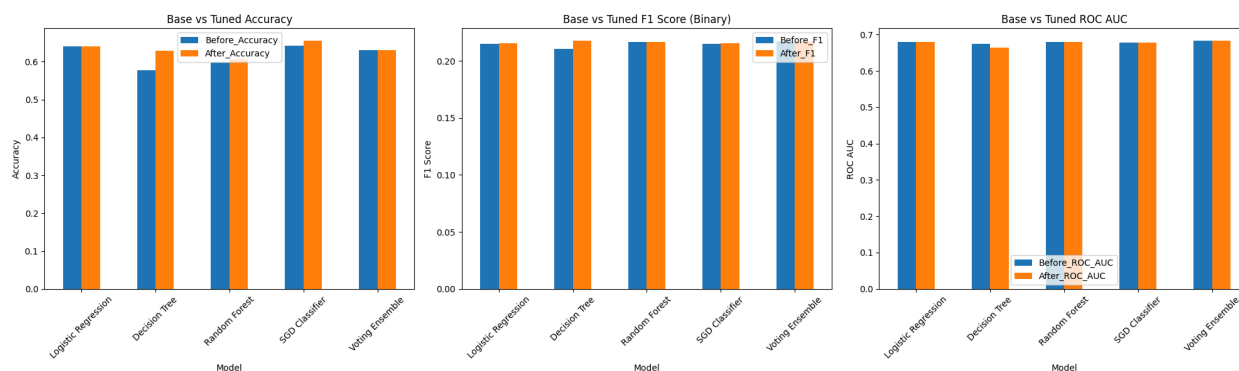
contain very few fraud cases making evaluation unreliable. I used F1 Binary instead of accuracy since even a dummy model has 91.8% accuracy. But I included it for reference.

Section 4: Model Selection

I chose models we learned in class mainly to show my understanding of what I learned, plus for the following reasons. I chose Logistic regression because it is a simple, fast, and easily interpretable linear model. We use it in class for binary classification, and it gives probability estimates such as ROC, Area Under Curve, and for soft voting in an ensemble. I used `class_weight='balanced'` to automatically adjust class imbalance by giving more weight to the minority class. Its main weakness is that it assumes a linear decision boundary, which may underfit the complex, non-linear patterns often found in fraud data. A decision tree is a nonlinear model that minimizes Gini impurity. It captures the interactions between features naturally. However, decision trees are prone to overfitting if they are allowed to grow too deep. I planned to tune its depth to control this. I also included Random Forest, which is an ensemble of many decision trees. It handles non-linearity very well and performs well on tabular data. Its main weakness is that it is less interpretable than a single tree. SGD Classifier is a scalable linear model and it is very efficient for large datasets. Voting Ensemble combines predictions using soft voting. It improved stability by averaging probabilities. Additionally, I showed SVC(RBF Kernel) in # comments. I did this because I want to show its ineffectiveness in classification tasks for large dataset size. It took me a long time to train the model compared to the other model. Hence, it is not efficient for my dataset.

Section 5: Model Training, Hyperparameter Tuning, and Model Building

Logistic Regression and SGDClassifier models minimize log loss (cross-entropy). By using `class_weight='balanced'`, the loss function is modified to penalize mistakes on the minority (fraud) class more heavily. Decision Tree and Random Forest models minimize Gini impurity (or entropy). The `class_weight='balanced'` parameter adjusts the node impurity calculations to give more weight to fraud cases. And voting ensemble has no separate loss function. It basically combines the predicted probabilities from the base models by averaging them. This is called soft voting. I used a 5-fold stratified cross-validation because my dataset is highly imbalanced, and stratification makes each fold preserve the original class distribution. 5 folds means it repeats the process 5 times to balance bias and randomness. Hence, using cross-validation, as I learned in class, gives a stronger estimate of model performance than a single train/validation split, reducing the chance of overfitting a particular validation set. For hyperparameter tuning, I used GridSearchCV using a small grid to make it manageable. For logistic regression, `C` (inverse regularization strength) has a value [0.01, 0.1, 1, 10], decision tree's max depth has [4, 6, 8, 10] and min_samples_split [2,10,20], and random forest's n_estimators have [100, 200] and max depth of [6, 10, None]



This graph shows that tuning has slightly improved the F1 score of most of the models. It is most evident in Decision trees, which turned out to have the highest F1 Score. The graph also shows a slight improvement in accuracy and ROC AUC score.

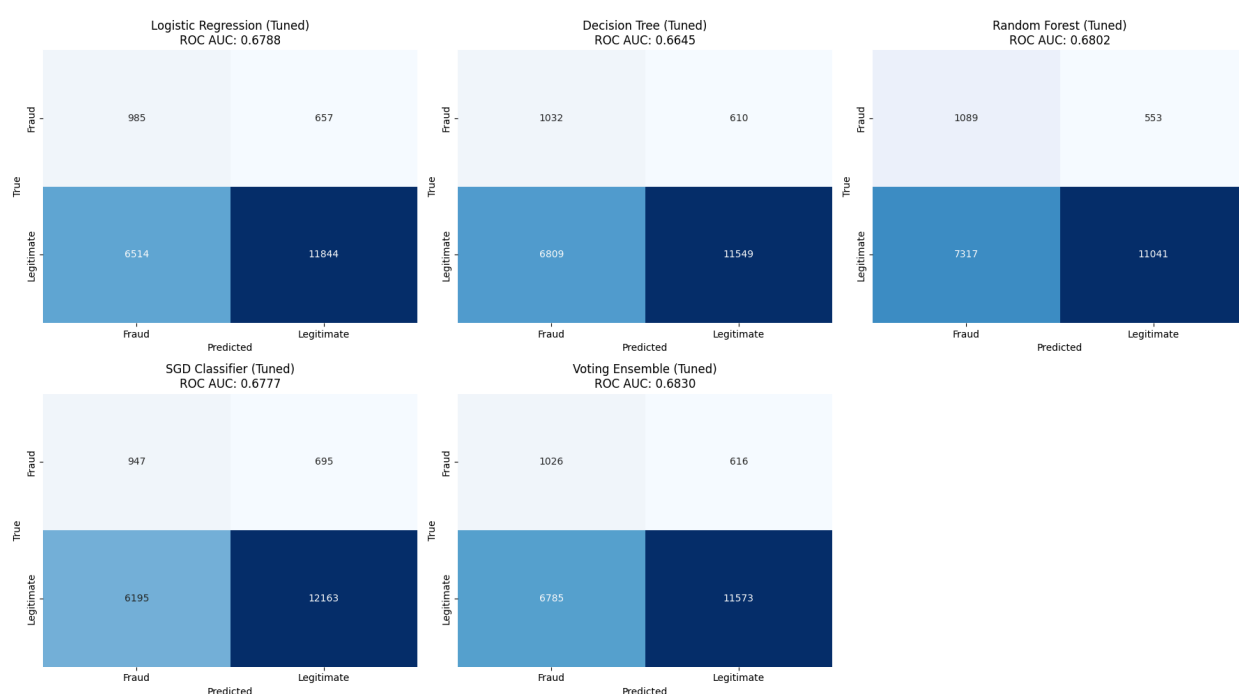
Section 6: Model Performance Metrics

To measure my models' performance, I used the F1 score. This is because in fraud detection, the only class that truly matters is the fraudulent one. Using `f1_weighted` would be misleading because the dataset is imbalanced. `f1_weighted` gives more weight to the majority class. So even if fraud detection was weak, the score would look high ($\sim 0.90+$), hiding the real problem. Additionally, `f1_macro` would also be misleading because Macro averages both classes equally. So I chose Binary F1, which directly measures how well I catch fraud, balancing precision and recall for class 1. I am also using ROC AUC to measure my model's ability to discriminate between classes across all possible classification thresholds. It is used for the imbalance class and gives a good overall picture of my model's quality. I included accuracy just for reference only. I knew from the start that it would be misleading due to the imbalance. Additionally, I used Test Time to measure performance because it allows me to compare test times to understand the efficiency of the models.

--- Final Model Performance Comparison ---							
	Model	Before_Accuracy	Before_F1	Before_ROC_AUC	After_Accuracy	After_F1	After_ROC_AUC
0	Logistic Regression	0.6411	0.2152	0.6788	0.6414	0.2155	0.6788
1	Decision Tree	0.5776	0.2108	0.6739	0.6290	0.2177	0.6645
2	Random Forest	0.5982	0.2166	0.6801	0.6065	0.2168	0.6802
3	SGD Classifier	0.6428	0.2149	0.6777	0.6555	0.2156	0.6777
4	Voting Ensemble	0.6300	0.2171	0.6830	0.6300	0.2171	0.6830

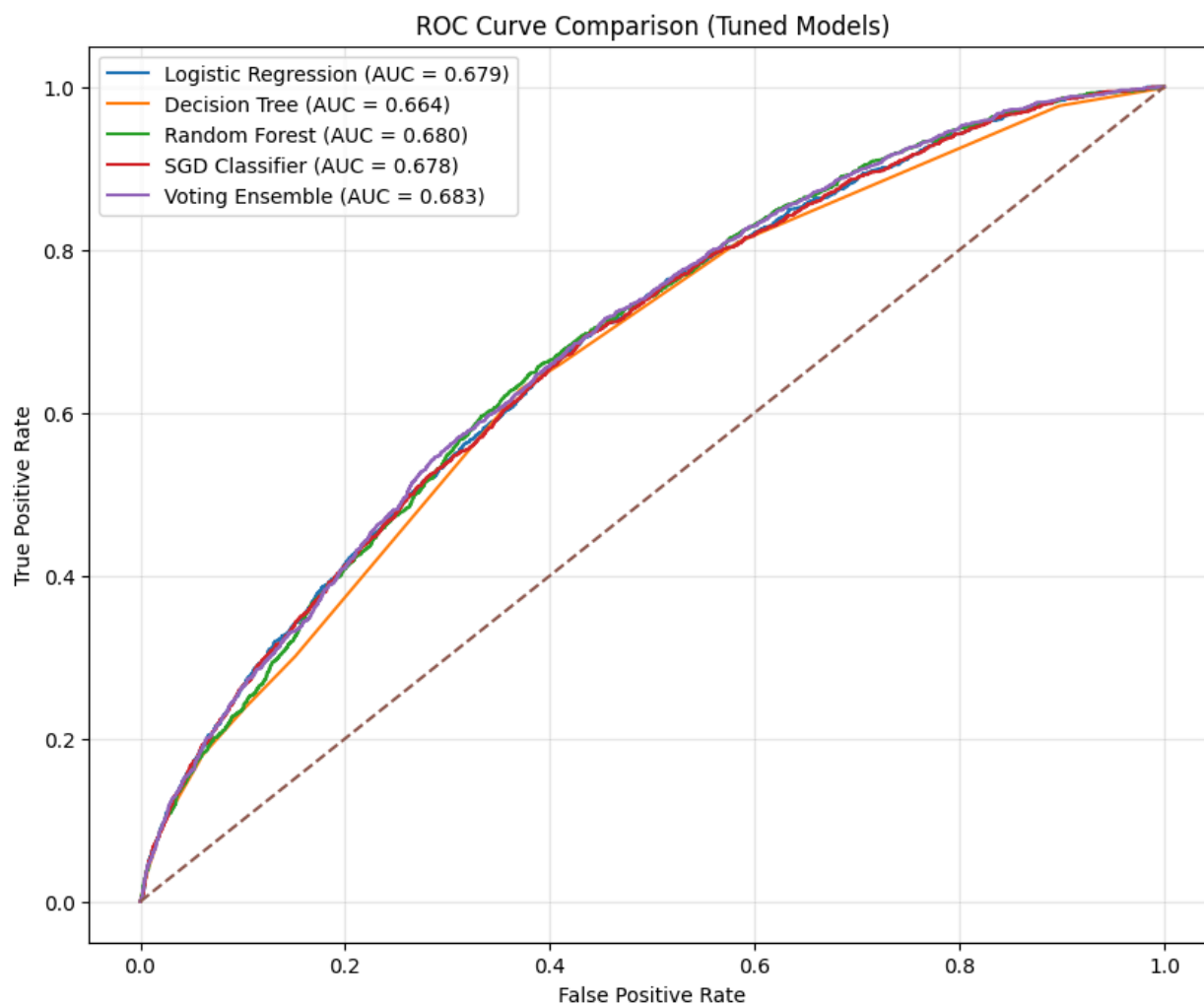
This shows us that improvement is possible with tuning. It brought modest improvements in my models. The Decision Tree gained the most, with its F1 increasing from 0.2108 to 0.2177. The Decision tree achieved the highest tuned F1 (0.2177), and the voting ensemble achieved the highest ROC AUC (0.6830).

Section 7: Results Visualization and Discussion



The confusion matrices show that the models are able to detect a meaningful portion of fraud cases, but performance is still limited. For example, the decision tree correctly identifies 1,032 out of 1,642 fraud cases in the test set (True Positives), while missing 610 fraud cases (False Negatives). Although more than half of fraudulent transactions are detected, a significant number are still missed. False negatives are the most costly error in fraud detection because they directly result in financial loss. The number of false positives is also relatively high. For example, the decision tree falsely flags 6,809 transactions as fraud. This is very dangerous.

Adding strong predictive features increases the real accuracy and F1-score, but realistically, can not be 100%.



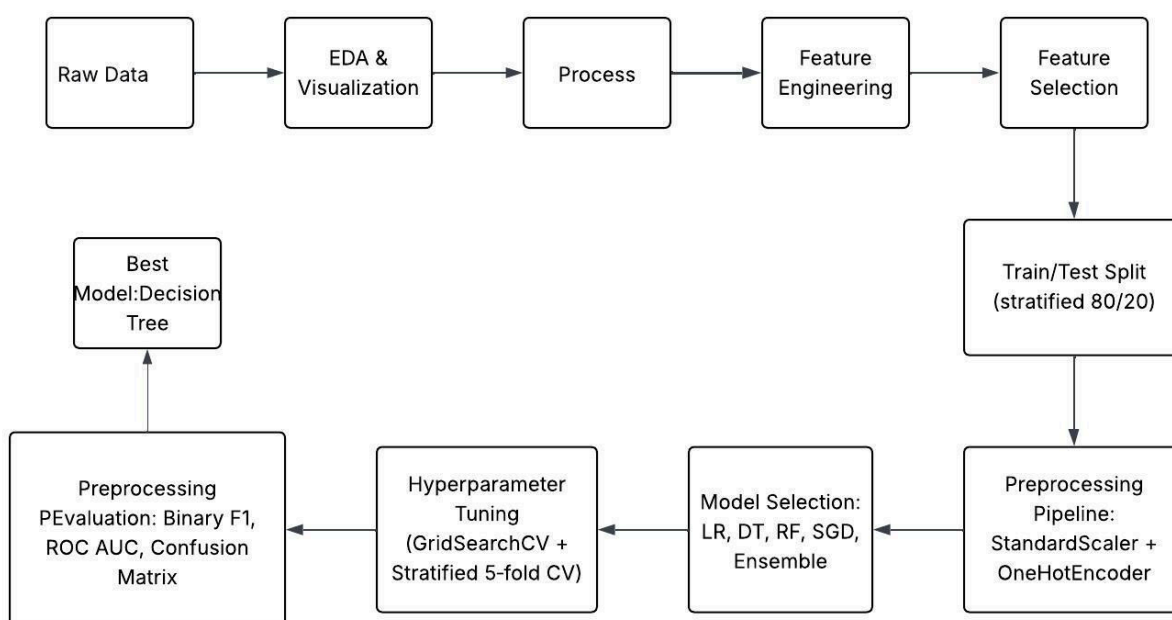
All curves lie close together, with AUC values around 0.68. This confirms that no single model dramatically outperforms the others. It shows the lack of a clear pattern for separating the two classes using just features.

Section 8: Summary

The three main things I learned from this assignment are that first, accuracy in imbalanced classification is misleading. For example, a model achieving 91.8% accuracy is

useless if it catches zero fraud. Hence, I learned how to handle similar cases by focusing on other performance metrics. Second, I learned how important the validity of a dataset is. I learned this the hard way. I initially was working on a synthetic dataset about cancer. The dataset is fundamentally flawed. For example, it assigns male-only cancers like testicular cancer to females, and it was random and has no pattern. Hence, the dataset is very, very important. Third, I learned voting ensemble is usually better. Even though a decision tree is better in my current work, it usually outperforms individual models. I learned that voting can give stronger predictions, even when the base models are weak.

Here is the complete ML pipeline diagram



The strength of my assignment is mainly the handling of class imbalance. I used `class_weight = 'balanced'` and stratified cross-validation, and I focused on binary F1. Second, I showed five models with consistent preprocessing and tuning. The main weakness of my assignment is that an F1 of ~ 0.22 is too low for practical use, but I tried to show my concept. I tried using other models we didn't learn in class, such as KNN and XGBoost, but I didn't get any

F1 scores that were significantly higher with the 8 features. I could have used some other highly correlated features like `fraud_flag_new_account`, `fraud_flag_ip`, and similar `fraud_flag_ip` but this would create me data leakage. I could have easily used them to get an inflated F1 score. My other weakness could be not exploring other advanced techniques, such as SMOTE or gradient boosting, which could potentially improve results.